Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación

IIC2233 Programación Avanzada (2025-1)

Tarea 2

Entrega

- **Tarea y** `README.md`
  - **Fecha y hora oficial (sin atraso): 17 de abril**
  - **Fecha y hora máxima (2 días de atraso): 19 de abril**
  - **Lugar:** Repositorio personal de GitHub — Carpeta: `Tareas/T2/`.
    El código debe estar en la rama (*branch*) por defecto del repositorio: `main`.
  - **Pauta de corrección**: en este enlace.
  - **Bases generales de tareas (descuentos)**: en este enlace.
- **Ejecución de tarea**: La tarea será ejecutada **únicamente** desde la terminal del computador. Además, durante el proceso de corrección, se cambiará el nombre de la carpeta "`T2/`" por otro nombre y se ubicará la terminal dentro de dicha carpeta antes de ejecutar la tarea. **Los *paths* relativos utilizados en la tarea deben ser coherentes con esta instrucción.**

Objetivos

- Aplicar conceptos de programación orientada a objetos (POO) para modelar y resolver un problema complejo.
- Utilizar decoradores, *properties* y clases abstractas como herramientas de modelación.
- Procesar *input* del usuario de forma robusta, manejando potenciales errores de formato.
- Trabajar con archivos en varios formatos para leer y procesar datos.
- Familiarizarse con el proceso de entrega de tareas y uso de buenas prácticas de programación.

Índice

1. Resumen ejecutivo **3**

2. *DCConquista de Yggdrasil* **3**

3. Flujo del programa **4**
3.1. Ejecución 4
3.2. Selección inicial 4
3.3. Juego 5

4. Menús **6**
4.1. Menú principal 6
4.2. Tienda 7

5. Entidades **7**
5.1. Árbol 7
5.2. Rama 8
5.2.1. Efectos adicionales 10
5.3. Modificadores 10
5.3.1. Modificadores positivos 10
5.3.2. Modificadores negativos 10

6. Combates **11**
6.1. Selección de atacantes 11
6.2. Cálculo de daño emitido 11
6.3. Recepción de daño 12

7. Archivos **12**
7.1. Árboles por dificultad 12
7.2. Ramas Disponibles 13
7.3. Modificadores positivos 14
7.4. Modificadores Negativos 14
7.5. parametros.py 15

8. Bonus: Guardar partida (+5 décimas) **16**

9. `.gitignore` **16**

10. Importante: Corrección de la tarea **17**

11. Restricciones y alcances **17**

1. Resumen ejecutivo

En esta tarea se solicita que desarrolles un programa en **Python** que simule un juego solitario, es decir, de un solo jugador. Se trata de un juego de combate por rondas en que el usuario compite contra la computadora. La ejecución e interacción del programa se debe realizar en su totalidad a través de la terminal del sistema, y la entrega de la Tarea debe realizarse en tu repositorio personal.

2. *DCConquista de Yggdrasil*

Con el pasar de los meses, tu dedicación al arte del bonsái y al desarrollo de *DCCortaRamas* te llevó a perfeccionar la simulación de poda y equilibrio arbóreo. Sin embargo, algo inesperado ocurrió. Uno de los árboles digitales dentro de tu simulador creció más allá de los límites establecidos. Sus raíces se expandieron por el código, sus ramas se multiplicaron en fractales imposibles y, antes de que pudieras detenerlo, había evolucionado en algo más... había nacido **Yggdrasil.exe**, el árbol corrupto.

Figura 1: Logo de *DCConquista de Yggdrasil*

Será necesario crear un programa jugable capaz de hacerle frente a esta entidad malvada que corroe los archivos de las plantas digitales. Crea el juego, elige un bonsái guerrero y enfréntate a su maldad incontenible. La última batalla está cerca, **Yggdrasil.exe** se cierne sobre la estabilidad de todos los sitemas computacionales. Si triunfas, se podrá restaurar el equilibrio de las plantas binarias. Si pierdes... tendrás que empezar a buscar otro pasatiempo. El destino del universo digital del bonsái está en tus manos.

3. Flujo del programa

DCConquista de Yggdrasil es un juego donde se debe manejar la existencia e interacción entre diversas entidades en un flujo ordenado, por lo que se ofrece la siguiente vista general de las partes con que se compone dicho flujo, y lo que cada parte implica:

Parte I: Inicio del programa

1. Ejecución del programa con la dificultad como argumento (`facil`, `normal` ó `dificil`)
2. Interfaz de selección de árbol combatiente utilizado por el jugador

Parte II: Fase intermedia

1. Presentar menús para que el usuario pueda preparar y analizar el estado del juego.
2. Permitir la compra de modificadores para mejorar las ramas del árbol seleccionado.
3. Permitir que el usuario vea el estado del árbol y todas las ramas.
4. Permitir iniciar el combate o pasar de ronda.

Parte III: Combate y pasar ronda

1. Si el jugador decidió atacar, se deben aplicar las mecánicas de combate.
2. Se deben aplicar las mecánicas de transcurso de ronda.

Parte IV: Fin del juego

1. Manejar correctamente el fin del juego, presentando la información mínima solicitada.

3.1. Ejecución

En esta tarea, deberás programar un juego de batalla y estrategia en que el jugador administrará un Árbol guerrero, y deberá tomar decisiones estratégicas para fortalecerlo y derrotar a **Yggdrasil.exe**. Al iniciar el programa, el jugador debe ingresar la dificultad del juego como un argumento en la terminal. Las opciones de dificultad son: `facil`, `normal` y `dificil`. Para manejar argumentos desde la línea de comandos, puedes investigar sobre el uso de `sys.argv` en internet. Este es un ejemplo de cómo puede estructurarse un comando para ejecutar el juego: `python3 main.py dificil`

Toda la interacción con el programa se debe realizar exclusivamente a través de la consola. Esto incluye la presentación de la información, el ingreso de las elecciones del usuario, y las respuestas del programa. Se evaluará la robustez en la interacción de tu programa, esto quiere decir que todos los menús deben ser a prueba de cualquier tipo de error; si se ingresa una opción inválida, el programa debe informar sobre el error y volver a mostrar las opciones del menú hasta que se ingrese una opción válida.

3.2. Selección inicial

Una vez establecida la dificultad, se ingresa a la Selección inicial, donde el jugador debe elegir el árbol con que jugará. Cada Árbol está compuesto por varias instancias de tipo Rama. Las ramas poseen atributos tales como vitalidad, resistencia y ataque. Todas las ramas pueden ser equipadas con Modificadores positivos, que modifican sus estadísticas básicas. Además, algunas ramas tienen Efectos adicionales, que deben aplicarse según la situación lo requiera.

Por todo lo anterior, es importante que el usuario tenga la posibilidad de seleccionar su árbol guerrero. Las opciones de guerreros que el jugador puede elegir son establecidas en archivos que serán entregados con la tarea. En esta interfaz se deben mostrar todas las opciones que se encuentran definidas en esos

recursos. El archivo particular desde el que se cargan los guerreros disponibles depende de la dificultad. Todo esto es explicado con mayor detalle en la sección Archivos.

Por ejemplo, si un archivo de dificultad contiene los árboles: **Roble Fortachón**, **Secuoya Rascacielos** y **Espino Espinoso**; el menú deberá mostrar el nombre de cada árbol junto con su cantidad de ramas. En la Figura 2 se muestra una presentación de la interfaz descrita.[1]

```
----------------------------
       SELECCIÓN INICIAL
----------------------------
[1] Roble Fortachón :       8 Ramas
[2] Secuoya Rascacielos :   12 Ramas
[3] Espino Espinoso :       5 Ramas

Selecciona tu árbol guerrero:
```

Figura 2: Ejemplo de interfaz de selección inicial

Una vez que el jugador haga su elección, el programa debe automáticamente elegir de forma aleatoria un árbol para el oponente, esta elección puede incluso ser una repitición de la elección del usuario. Este será el árbol enemigo al que se enfrente el usuario, y será controlado por el programa. Antes de continuar a la siguiente fase, se debe mostrar un cuadro de información resumen con el árbol seleccionado y el árbol oponente, junto con un mensaje de bienvenida.

Consideraciones importantes de la selección inicial

- El menú solo debe mostrar las opciones de árboles que estén disponibles en el archivo cargado.
- Una vez seleccionado el árbol, no será posible regresar a esta interfaz. Se debe dirigir de forma automática al Menú principal.
- Esta fase de selección inicial es obligatoria y no se puede iniciar un juego nuevo sin que se complete la elección.

3.3. Juego

Tal como se mencionó, después de elegir el árbol, se debe ingresar al Menú principal, donde el jugador podrá administrar la partida. Esta se inicia con un valor base de dinero `DINERO_INICIAL`[2], que se utiliza para comprar modificadores en la Tienda.

El juego se desarrolla en rondas, en cada una el jugador puede realizar cuantas acciones desee, siempre respetando las reglas asociadas a cada acción. El jugador puede decidir **atacar**, lo que da inicio a un combate y pasa la ronda al final de éste (véase Combates), ó puede decidir **pasar la ronda sin atacar**.

Al pasar la ronda se recibe un `GANANCIA_POR_RONDA` de dinero, se activan los efectos de ramas y modificadores que correspondan según su descripción y, además, se debe recorrer cada rama del árbol para determinar si a ellas les aparece algunos de los Modificadores negativos. El orden de estos eventos es importante, y debe seguir el siguiente flujo[3]:

[1] Los nombres y la cantidad de ramas que aparecen en este menú (y los otros menús) son solo ejemplos y no reflejan necesariamente el contenido real de los archivos.

[2] A lo largo del enunciado encontrarás varias palabras escritas en `ESTE_FORMATO`. Estas palabras corresponden a parámetros y puedes encontrar los detalles sobre ellos en la sección `parametros.py`

[3] Para cada uno de estos eventos y etapas del transcurso de la ronda, se debe imprimir en consola la información relacionada.

1. Aplicar todas las mecánicas de combate (solo si hubo combate en esta ronda).
2. Aplicar efectos de las ramas que correspondan según su descripción.
3. Obtención de `GANANCIA_POR_RONDA` dinero.
4. Recorrer ramas y determinar si les aparece un Modificadores negativos.

Luego de cada transcurso de ronda, si ambos árboles sobreviven, se debe regresar al Menú principal para continuar la partida. Si la **rama principal** del árbol guerrero del jugador es destruida, se pierde la partida y se muestra un mensaje de derrota. Por otro lado, si el jugador destruye la rama principal del árbol enemigo, se se debe un mensaje de victoria. En ambos casos, el juego finaliza automáticamente.

4. Menús

El flujo del juego debe ser manejado a través de menús con los que el jugador interactúa para indicar las acciones a realizar. El programa debe tener a lo menos los dos menús descritos en esta sección, pero puedes agregar más si lo deseas, siempre respetando las condiciones y el contenido mínimo de las interfaces aquí descritas. Los menús que el programa debe tener son: Menú principal, y Tienda.

4.1. Menú principal

```
-----------------------------
       MENÚ PRINCIPAL
-----------------------------
Dinero disponible: $16
Ronda Actual: 1

[1] Atacar y pasar ronda
[2] Pasar ronda sin atacar
[3] Tienda
[4] Ver información del árbol
[5] Espiar al árbol enemigo
[6] ** Guardar partida
[7] Salir del juego

Indique su opción:
```

Figura 3: Ejemplo de Menú Principal

Es el menú más importante, que provee al usuario de todas las acciones para manejar el juego. El jugador debe poder regresar a este desde cada interfaz y sub-menú del programa. En la figura 3 se presenta una posible visualización para el menú. Los elementos mínimos que debe tener son:

- **Atacar y pasar ronda**: se deben presentar las dos opciones asociadas: **atacar y pasar ronda**, y **pasar ronda sin atacar**. Si se selecciona la primera, se le debe preguntar al jugador con qué rama de su árbol guerrero desea atacar, y luego ejecutar todas las mecánicas de combate respectivas, de acuerdo con lo que se describe en Combates.

  Luego, se deben ejecutar las acciones relacionadas al paso de ronda en el orden en que estas tienen ocurrencia. Esto se describe en Juego.

- **Tienda**: debe redirigir al menú Tienda.
- **Ver información del árbol**: el programa debe preguntarle al jugador si quiere obtener la información completa del `Arbol`, o un resumen. La información completa del árbol debe mostrar el

nombre de árbol, incluyendo sus `ramas`, sus `sub-ramas` y las estadísticas individuales de cada una. Un ejemplo de cómo se puede presentar esta información es visible en la Figura 5.

- **Espiar al árbol enemigo**: se debe imprimir información general del árbol enemigo. En este caso no existe la opción de obtener detalles específicos de cada una de las ramas. Solo se dispone de la información de resumen.
- **** Guardar partida**: Este elemento del Menú Principal es opcional, y los detalles de su implementación se encuentran en la sección Bonus: Guardar partida (+5 décimas)
- **Salir del juego**: se imprimirá un mensaje de despedida y se cerrará el programa.

4.2. Tienda

En este menú el jugador tiene la opción de adquirir nuevos modificadores para colocar en su propio árbol en la rama que desee. Los modificadores disponibles dependen del archivo Modificadores positivos, y estos representan un catálogo de opciones. Cuando se compra un modificador, este no desaparecerá de las opciones de la tienda, por lo que podrá ser seleccionado nuevamente siempre que el usuario tenga suficiente dinero. El modificador podrá ser adquirido tantas veces como se desee.

El menú debe desplegar todos los modificadores disponibles junto a sus precios, y la información del dinero actual del jugador. Cuando el jugador compra un modificador, este debe ser adjunto de inmediato a alguna rama, por lo que es necesario que se le pregunte al jugador en qué rama desea posicionarlo. La asignación de un modificador a una rama debe seguir las reglas descritas en Rama. En la figura 4 se presenta una posible visualización del menú descrito.

```
-----------------------------
            TIENDA
-----------------------------
Modificador:         Precio:
[1] Naranja jugosa       $10
[2] Musgo de acero        $5
[3] Ardilla agresiva      $6
[4] Trampa de resina      $2
[5] Volver al Menú principal

Indique su opción:
```

Figura 4: Ejemplo de Menú Tienda

5. Entidades

En esta sección se detallarán las distintas entidades que deberás implementar para modelar *DCConquista de Yggdrasil*. Para ello, tendrás que utilizar conceptos clave de la **Programación Orientada a Objetos** como herencia, *properties*, clases abstractas y polimorfismo. Ten en cuenta que cada uno de estos conceptos debe ser incluido en la tarea **al menos una vez**, por lo que deberás descubrir dónde implementarlos **correctamente** según lo propuesto en el enunciado.

Las tres entidades principales de *DCConquista de Yggdrasil* son **Árbol**, **Rama** y **Modificadores**. Debes incluir **como mínimo** las características nombradas a continuación, pero siéntete libre de añadir nuevos atributos y métodos si lo estimas necesario.

5.1. Árbol

El `Arbol` es la entidad principal en *DCConquista de Yggdrasil*. Es el guerrero que atacará desde sus ramas hacia el enemigo **Yggdrasil.exe**. Para lograr el objetivo, el árbol debe resistir a los ataques enemigos y

destruir su rama principal, por lo que la estrategia es clave para alcanzar la victoria. `Arbol` debe tener, al menos, los siguientes atributos:

- **Rama Principal**: Contiene la `Rama` principal del árbol, desde aquí se expanden todas las ramas hijas.
- **Nombre**: Contiene el nombre del árbol.

Además, debe tener por lo menos los siguientes métodos:

- **Cargar modificador**: Debe recibir una entidad `Rama` y una entidad `Modificador`. Debe buscar esa rama indicada y asignarle el modificador entregado.
- **Atacar**: Debe recibir una entidad `Rama`, que corresponde a la selección del usuario, y debe calcular el ataque que se genera al atacar desde esa rama. Para ello se debe aplicar el método descrito en Combates. En el caso de **Yggdrasil.exe**, la selección debe ser aleatoria.
- **Pasar ronda**: En este método, el árbol debe encargarse de aplicar los efectos que estén asociados al paso de una ronda, llamando a los métodos respectivos de cada `Rama`. En este método se debe además administrar, por cada `Rama` del árbol, la posible aparición de Modificadores negativos.
- **Resumir Árbol**: En este método el árbol debe retornar un *string* que contenga la información resumida de su contenido. Es decir, debe calcular la vida total del árbol como la suma de la vitalidad de todas las ramas hijas. El ataque promedio, considerando el ataque de todas sus ramas. La cantidad de ramas que tiene el árbol, y su profundidad máxima. Puedes incluir cualquier otra información que consideres relevante.
- **Presentarse**: En este método debes sobrescribir los métodos de clase `__str__` y `__repr__` de modo que `Arbol` se imprima con toda su estructura detallada, incluyendo todas sus ramas y atributos. Puedes ver en la Figura 5 un ejemplo de cómo podría verse el resultado de este método. En el caso del árbol enemigo, esta información no es visible para el jugador, por lo que este método debe retornar lo mismo que **Resumir Árbol**.

```
--------------
Roble Guerrero
--------------
   Cactoos: Puntos de vida: 90, Resistencia: 0.1, Daño base: 120
       \--subramas-
        rama veloz: Puntos de vida: 80, Resistencia: 0.34, Daño base: 90
        Paalm: Puntos de vida: 85, Resistencia: 0.21, Daño base: 110
           \--subramas-
           cactoos: Puntos de vida: 50, Resistencia: 0.11, Daño base: 100
           Ramita: Puntos de vida: 80, Resistencia: 0.4, Daño base: 140

    Ficus: Puntos de vida: 100, Resistencia: 0.4, Daño base: 150
      \--subramas-
       Rama inteligente: Puntos de vida: 30, Resistencia: -0.2, Daño base: 60
```

Figura 5: Ejemplo de presentación completa de `Arbol`

5.2. Rama

Las ramas son la composición de los árboles en *DCConquista de Yggdrasil*. Estas poseen diversas estadísticas que dictaminan qué tan poderosos serán los ataques de cada `Arbol` y cómo estos resistirán a las arremetidas de su rival.

La entidad `Rama` debe tener al menos los siguientes atributos:

- **Nombre**: Contiene el nombre de la rama.
- **Vitalidad Máxima**: Corresponde a un `int` que representa cantidad máxima de puntos de vida que puede tener la rama.
- **Efecto**: Un `string` con la descripción del efecto propio de la rama. Véase Efectos adicionales.
- **Salud**: Corresponde a un `int` entre 0 y **Vitalidad Máxima** que representa la cantidad de vida actual de la rama. Cuando los puntos de salud de una rama llegan a 0, esta debe morir, y con ella, todas sus ramas hijas.
- **Daño base**: Corresponde a un `int` que representa la cantidad de puntos de daño que inflinge la rama. Este número no considera los posibles efectos o aumentos que provengan de otras entidades.
- **Defensa**: Corresponde a un `float` entre -0.5 y 0.5 que representa la reducción de daño de la rama al recibir daño.
- **Resistencia a plagas**: Corresponde a un `float` entre 0 y 1 que representa la resistencia a la aparición de un Modificadores negativos al pasar de ronda. Por ejemplo, si una `Rama` tiene resistencia a plaga de 0.8, entonces tiene un 80 % de probabilidades de que no le aparezca una `Plaga` durante el paso de una ronda.
- **Modificador**: Si lo hay, corresponde al objeto de la clase `Modificador` equipado en la rama. Por lo que su valor puede ser `None` ó una instancia de `Modificador`. En caso de que la rama tenga un `Modificador` equipado, se mantiene el bonus o descuento aplicado por el objeto **solo mientras permanezca en la rama**. Si una rama tiene un `Modificador` equipado, la única forma de retirarlo es reemplazándole por otro.[4]
- **Ramas hijas**: Corresponden a todas las ramas que derivan de la rama actual.

La entidad `Rama` debe tener al menos los siguientes métodos:

- **Cargar modificador**: Permite a la rama equiparse un objeto `Modificador`. La rama adquiere el bonus o descuento del objeto cargado. Si la rama ya tenía un modificador cargado, ese debe ser descartado (eliminado) y el nuevo que se está intentando asignar debe permanecer.
- **Pasar Ronda**: Actualiza el estado de la rama, si esta posee un efecto asociado al paso de ronda, este se debe aplicar. Además, se debe verificar si en esta ronda le aparece un modificador negativo (véase Modificadores negativos), esto quiere decir, que se debe hacer una verificación de probabilidad considerando su **Resistencia a plagas** y, de fallar la resistencia, se deberá cargar un modificador negativo seleccionado de forma aleatoria en la rama.
- **Atacar**: Retorna los puntos de daño que ejerce la rama. Esto debe considerar la influencia que pueda tener el efecto de la rama y/o el modificador equipado.
- **Recibir daño**: Recibe un `int` que representa puntos de daño entrantes, luego, debe calcular el daño verdadero aplicado al considerar la defensa de la rama y sus posibles efectos. Debe descontar de su **Salud** el daño verdadero recibido. Finalmente, debe retornar los puntos de daño "sobrantes", que pasarán a ser aplicados a su rama padre. Véase Combates.
- **Presentarse**: En este método debes sobrescribir los métodos de clase `__str__` y `__repr__` de modo que la rama se imprima con toda su información, incluyendo sus atributos de juego (**Nombre**, **Salud**, **Vitalidad Máxima**, **Daño base**, **Defensa**, **Resistencia a plagas**, **Modificador equipado**, y número de ramas hijas). En la Figura 6 se muestra un ejemplo de cómo puede lucir el resultado de este método.

[4] Si una rama tiene un modificador cuyo bonus es entregar vitalidad máxima, y este modificador es lo único que mantiene a la rama con vida, entonces al retirarlo, la rama muere.

```
Cactoos: Puntos de vida: 570/800, Defensa: 10%, Daño base: 68, Resistencia a Plagas: 50%
```

Figura 6: Ejemplo de presentación de rama

5.2.1. Efectos adicionales

Existen distintos tipos de ramas, cada una posee un efecto especial, estos efectos se aplican en diferentes contextos o fases de la ejecución del juego. Los tipos de ramas y sus efectos se detallan en el Cuadro 1.

Rama	**Efecto**
Ficus	Cada vez que transcurre una ronda, esta rama y sus hijas recuperan 4 % de la salud faltante.
Celery	Por cada ronda que esta rama no ataque ni forme parte de un ataque, su daño base aumenta un 1 %.
Hyedrid	Esta rama se puede equipar dos objetos `Modificador` en lugar de uno. Para la mecánica de reemplazo, se debe considerar el órden en que ingresan los objetos. Debe seguir una secuencia **FIFO**.
Paalm	Cada vez que esta rama recibe daño y no muere, su defensa aumenta en 0.02.
Alovelis	Sin efectos adicionales.
Pine	Si esta rama muere por un ataque, anula el daño de penetración hacia su padre.
Cactoos	Si no ataca ni recibe daño durante una ronda, genera `DINERO_CACTOOS` de dinero por cada rama hija.

Cuadro 1: Tipos de ramas y sus efectos.

5.3. Modificadores

Los modificadores en *DCConquista de Yggdrasil* juegan un papel fundamental para potenciar las ramas de los árboles. Estás son entidades que se pueden asociar ó equipar a cada rama y, al hacerlo, modifican las estadísticas básicas de esta. Los modificadores se clasifican en dos categorías: **Modificadores positivos** y **Modificadores negativos**.

5.3.1. Modificadores positivos

Pueden ser comprados en la Tienda. Este tipo de modificador aplica un bonus de características básicas a la rama que tenga equipado el objeto. Estos bonuses son por lo general positivos, pero en ocasiones puede haber un modificador que sacrifica cierto tipo de característica para potenciar con mayor eficacia alguna otra. Por ejemplo, un modificador que disminuya la defensa en 0.2 para aumentar el daño base en 100. Los modificadores de este tipo que el jugador puede comprar se encuentran listados en el archivo respectivo. Véase Modificadores positivos.

5.3.2. Modificadores negativos

Este tipo de modificadores no se pueden comprar. En su lugar, intentan aparecer en las ramas durante cada ronda que pasa. Dependerá de la característica **Resistencia a plagas** de cada rama si finalmente aparece, o no, un modificador de este tipo.

Como se trata de un modificador, este se carga en la rama retirando al modificador que pudiera tener anteriormente dicha rama. Los modificadores de este tipo que pueden aparecer durante cada transcurso

de ronda se encuentran listados en el archivo respectivo. Véase Modificadores negativos.

6. Combates

Como el objetivo del juego es derrotar al **Árbol Enemigo**, el combate es una de las partes centrales del programa. El **ataque** es una de las posibles acciones del jugador entre cada ronda. Cuando el jugador decide atacar, se deben realizar diferentes procesos y acciones que determinan cómo se afectan las entidades `Arbol` y sus `Ramas`.

Un **combate** refiere a todo el proceso de intercambio de daño entre el `Arbol` del jugador y el `Arbol` enemigo. Este se divide en tres etapas:

6.1. Selección de atacantes

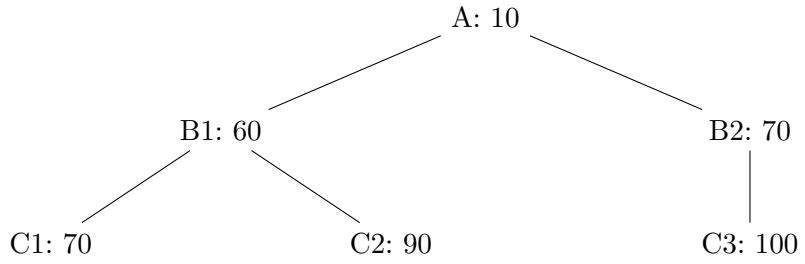
En esta etapa, el jugador debe seleccionar la rama de su propio `Arbol` con la que desea atacar. Esta debe ser una rama que se encuentre viva en el `Arbol`. Puede elegir cualquier rama sin importar su profundidad dentro del árbol.

En el caso del oponente, la selección de la rama es aleatoria, de modo que el programa debe determinar con qué rama ataca **Yggdrasil.exe**.

6.2. Cálculo de daño emitido

En esta etapa se debe calcular el daño que **emite** cada `Arbol`. Para calcular el daño, se debe obtener el promedio del daño de ataque entre la rama seleccionada, todas sus ramas hijas, y las hijas de las hijas, etc. La forma en que se calcula el daño **emitido** es la misma para ambos áboles en el juego. La única diferencia es que, para el caso de **Yggdrasil.exe**, la rama desde donde se inicia el ataque es elegida aleatoriamente.

Para ilustrar este cálculo, se presenta en la siguiente Figura un `Arbol` de profundidad 3 donde cada `Rama` está indicando el ataque que produce por sí sola.



De acuerdo con este diagrama se proponen como ejemplo las siguientes situaciones basadas en la elección del jugador:

- Situación 1: El jugador **elige la rama C3** para atacar. En este caso, el daño **emitido** es justamente 100, dado que C3 no tiene ramas hijas.
- Situación 2: El jugador **elige la rama B1** para atacar. En este caso, se debe considerar como daño **emitido** la suma de cada una de las ramas involucradas dividido en el número de ramas involucradas, la expresión queda de la siguiente forma:

  ```
  floor((B1.ataque + C1.ataque + C2.ataque) / 3) = 73
  ```

- Situación 3: El jugador **elige la rama A** para atacar. En este caso, se debe considerar como daño **emitido** la suma de todas las ramas del árbol, dividido en la cantidad de ramas del árbol. La expresión queda de la siguiente forma, considerando solo las magnitudes de daño para simplificar:

$$\texttt{floor((10 + 60 + 70 + 90 + 70 + 100) / 6) = 66}$$

6.3. Recepción de daño

Una vez que se ha calculado el daño emitido desde cada `Arbol`, se debe **aplicar** el daño de ambos luchadores. **Primero el árbol del jugador debe recibir el daño**, y luego, **Yggdrasil.exe** debe recibir el daño hecho por el guerrero del jugador. Considera que cualquier árbol podría morir antes de tener la posibilidad de atacar.

El ataque siempre será recibido por la rama más alta, es decir, la que tenga el mayor nivel de profundidad dentro del **Árbol**. En caso de tener 2 o más subramas al mismo nivel de profundidad, se debe elegir de forma aleatoria cual recibe el ataque.

El daño recibido por la rama, será el daño **emitido** por el árbol atacante, reducido en un porcentaje dado por la defensa de la `Rama` que recibe el ataque:

$$\texttt{daño_verdadero} = \text{round}\left(\texttt{daño_emitido} \times (1 - \texttt{defensa_rama})\right)$$

$$\texttt{salud} = (\texttt{salud} - \texttt{daño_verdadero})$$

En caso de que el daño verdadero que recibe una rama sea mayor a su salud, el daño sobrante debe traspasarse a la rama padre, donde se debe calcular el nuevo daño verdadero que se le restará a la rama padre.[5]

7. Archivos

Para el desarrollo de *DCConquista de Yggdrasil* será necesaria la lectura y carga de archivos. Estos contendrán información sumamente relevante para la ejecución correcta de tu programa.

7.1. Árboles por dificultad

Archivos de texto con los nombres `arboles_faciles.txt`, `arboles_medios.txt`, `arboles_dificiles.txt` ubicados en la carpeta `data/`, que definen la estructura de los arboles que el jugador y el enemigo podrán seleccionar según la dificultad seleccionada. Un ejemplo en la siguiente Figura 7.

```
Sauce Veloz: {Alovelis; {Pine; Celery; Paalm; Cactoos; Celery}; Hyedrid;
    Celery; Celery; Pine; Alovelis};
Fresno Lunar: {Alovelis; {Ficus; Celery; Pine; Paalm; Celery}; Ficus;
    Alovelis; Celery; Cactoos; Pine};
```

Figura 7: Archivo de ejemplo con árboles

[5] Recordar que si una rama muere, también deben hacerlo todas sus hijas (aunque estas nunca hayan recibido daño).

La Figura 8 explica mejor la estructura de los árboles, con sus respectivas ramas:

```
1. Sauce Veloz:
Alovelis
|-- Pine
|   |-- Celery
|   |-- Paalm
|   |-- Cactoos
|   \-- Celery
|-- Hyedrid
|-- Celery
|-- Celery
|-- Pine
\-- Alovelis

2. Fresno Lunar:
Alovelis
|-- Ficus
|   |-- Celery
|   |-- Pine
|   |-- Paalm
|   \-- Celery
|-- Ficus
|-- Alovelis
|-- Celery
|-- Cactoos
\-- Pine
```

Figura 8: Estructura de los árboles del archivo de ejemplo

Cabe destacar, cada árbol tiene un nombre único que lo identifica el cual aparece en el inicio de cada línea del archivo. Además, la estructura de los árboles contiene el nombre de cada una de sus ramas hijas, donde será tú tarea realizar la carga de las respectivas ramas hijas con sus atributos. En la siguiente sección se detallan los atributos de cada rama.

7.2. Ramas Disponibles

Archivo `ramas.txt` que define los tipos de ramas, con sus atributos y efectos ya mencionados en la sección Efectos adicionales. Podemos ver un ejemplo de los componentes en el Cuadro 2, y otro de un archivo de este tipo en la Figura 9.

Atributo	**Tipo**
`nombre_rama`	`string`
`puntos_de_vida`	`int`
`defensa`	`float` entre -0.5 y 0.5
`daño_base`	`int`
`resistencia_a_plagas`	`float` entre 0 y 1

Cuadro 2: Especificación de ramas

```
Ficus;1600;0.10;45;0.85
Celery;1200;0.00;35;0.75
Hyedrid;1000;-0.10;75;0.65
Paalm;2000;0.20;40;0.90
Alovelis;1800;0.00;25;0.80
Pine;900;-0.20;60;0.60
Cactoos;800;-0.30;55;0.50
```

Figura 9: Ejemplo de archivo `ramas.txt`

7.3. Modificadores positivos

Archivo `modificadores.txt` que define los modificadores disponibles en la tienda, donde cada uno aplicará bonificaciones a la rama que lo posea. Podemos ver un ejemplo de los componentes en el Cuadro 3, y otro de un archivo de este tipo en la Figura 10.

Atributo	**Tipo**
`nombre`	`string`
`ataque`	Alteración aplicada sobre el ataque de la rama
`defensa`	Alteración aplicada sobre la defensa de la rama
`vida_maxima`	Alteración aplicada sobre la vida máxima de la rama
`precio`	`int` que representa el costo de compra en la tienda

Cuadro 3: Especificación de los modificadores

```
Flor;0;0;50;50
Panal de Abejas;30;0;10;150
Nido de Pajaros;100;-0.1;0;200
Semilla Explosiva;100;-0.5;-200;400
```

Figura 10: Ejemplo de archivo `modificadores.txt`

7.4. Modificadores Negativos

Archivo `modificadores_negativos.txt` que define los modificadores negativos que pueden llegar a afectar a cualquiera de las ramas del árbol del jugador o del enemigo. Podemos ver un ejemplo de los componentes en el Cuadro 4, y otro de un archivo de este tipo en la Figura 11.

Atributo	**Tipo**
`nombre`	`string`
`ataque`	Fórmula aplicada (e.g. `"ataque + 2"`)
`defensa`	Fórmula aplicada (e.g. `"defensa + 2"`)
`vida_maxima`	Fórmula aplicada (e.g. `"vida_maxima + 2"`)

Cuadro 4: Especificación de modificadores

```
Termitas;0;-0.3;0
Zancudos;0;0;-100
Hongo Parasitario;0;-0.2;-50
Gusano Minador;-30;0;0
Moho Gris;0;-0.4;-100
Avispas Asesinas;-20;-0.1;-80
Ácaros Chupadores;0;0;-200
```

Figura 11: Ejemplo de archivo `modificadores_negativos.txt`

7.5. parametros.py

Para esta tarea, se requiere la creación de un archivo `parametros.py` donde deberás **completar todos los parámetros mencionados a lo largo del enunciado**. Dichos parámetros se presentarán en `ESTE_FORMATO` y en ese color. Además, es fundamental incluir cualquier valor constante necesario en tu tarea, así como cualquier tipo de *path* utilizado.

Un parámetro siempre tiene que estar nombrado de acuerdo a la función que cumplen, por lo tanto, asegúrate de que sean descriptivos y reconocibles. Ejemplos de parametrización:

```python
CINCO = 5  # mal parámetro
DINERO_MAXIMO = 3000  # buen parámetro
PROBABILIDAD_TORMENTA = 0.2  # buen parámetro
```

Si necesitas agregar algún parámetro que varíe de acuerdo a otros parámetros, una correcta parametrización sería la siguiente:

```python
PI = 3.14
RADIO_CIRCUNFERENCIA = 3
AREA_CIRCUNFERENCIA = PI * (RADIO_CIRCUNFERENCIA ** 2)
```

Dentro del archivo **parametros.py**, es obligatorio que hagas uso de todos los parámetros almacenados y los importes correctamente. Cualquier información no relacionada con parámetros almacenada en este archivo resultará en una penalización en tu nota. Recuerda que no se permite el **hard-coding**[6], ya que esta práctica se considera incorrecta y su uso conllevará una reducción en tu calificación.

[6] *Hard-coding* es la práctica de ingresar valores directamente en el código fuente del programa en lugar de parametrizar desde fuentes externas.

8. Bonus: Guardar partida (+5 décimas)

En esta sección se decribe una funcionalidad **opcional** para tu tarea. Su correcta implementación conlleva una puntuación extra a tu calificación de esta entrega. Para poder obtener este bonus debes cumplir con los siguientes requerimientos:

- La nota en tu tarea (sin bonus) debe ser **igual o superior a 4.0**.
- El bonus debe estar implementado **en su totalidad**, es decir, **no se dará puntaje intermedio**.
- Además, deberás explicitar en tu `README` si implementaste esta funcionalidad.

Lo que se solicita es que el programa ofrezca al jugador la opción de **Guardar partida** mientras se encuentra en el Menú principal. Debe habilitar la posibilidad de abandonar el juego **deteniendo el programa**, para reanudar la partida en otro momento.

Cuando el jugador decide **Guardar partida**, tu programa debe **generar un archivo** nuevo que contenga **toda la información** de la partida actual hasta ese momento, incluyendo los detalles del árbol tanto del jugador como de **Yggdrasil.exe**, la estructura de los árboles (sus ramas), la salud y todos los atributos de cada rama, la existencia y asociación de modificadores en las ramas, el número de rondas, el dinero actual, y toda otra información que sea relevante para el juego. Cuando se ha terminado de crear ese archivo, se le debe notificar al usuario que su partida ha sido guardada exitosamente.

El programa **no se detiene** cuando el jugador guarda la partida. El juego puede continuar normalmente, el jugador puede seguir atacando y pasando rondas cuantas veces lo desee, incluso podría terminar por completo el juego, ya sea al ganar o perder. Sin embargo, esto **no debe afectar** al estado de la partida que se guardó anteriormente.

El jugador puede guardar una partida cuantas veces desee, en cualquier momento en que se encuentre en el Menú principal, y para cada una de las veces que lo haga, deberás escribir la información del estado de la partida en el archivo mencionado. Queda a tu criterio si decides crear un archivo distinto para cada vez que se guarde una partida, o trabajar un solo archivo que se sobrescriba con cada guardado. En cualquier caso, el ó los nombres de estos archivos deben ser coherentes con el juego y deberás explicitar en tu `README` cómo se guarda la información de la partida dentro de estos archivos.

La recuperación de una partida previa se debe aplicar desde la ejecución del programa: en lugar de escribir la dificultad del juego como un argumento en la terminal, se debe escribir el **nombre del archivo** que contiene una partida previamente guardada. Es decir, tu programa deberá ser capaz de identificar correctamente las siguientes situaciones:

1. Situación 1: se ejecuta el programa entregando la dificultad (`facil`, `medio` ó `dificil`) como argumento en la terminal. En este caso, tu programa deberá iniciar una partida nueva y dirigir al jugador a la Selección inicial
2. Situación 2: se ejecuta el programa entregando **el nombre del archivo** que contiene una partida previamente guardada como argumento en la terminal. En este caso, tu programa deberá **cargar** los datos de esa partida, recuperando por completo el estado en que se encontraba el juego, y redirigir al Menú principal de dicha partida.

9. `.gitignore`

Para esta tarea **deberás utilizar un** `.gitignore` para ignorar los archivos indicados, este deberá estar dentro de tu carpeta `Tareas/T2/`.

Los elementos que no debes subir y **debes ignorar mediante el archivo** `.gitignore` para esta tarea son:

- El enunciado.
- La carpeta `data/`

Recuerda **no ignorar archivos vitales de tu tarea como los que tú creas o modificas, o tu tarea no podrá ser revisada**.

Es importante que hagan un correcto uso del archivo `.gitignore`, es decir, los archivos **deben** no subirse al repositorio debido al uso correcto del archivo `.gitignore` y no debido a otros medios.

10. Importante: Corrección de la tarea

En el **siguiente enlace** se encuentra la distribución de puntajes. En esta señalará con color **amarillo** cada ítem que será evaluado a nivel funcional y de código, es decir, aparte de que funcione, se revisará que el código esté bien confeccionado. Todo aquel que no esté pintado de amarillo será evaluado si y sólo si se puede probar con la ejecución de su tarea.

Importante: Todo ítem corregido por el cuerpo docente será evaluado únicamente de forma ternaria: cumple totalmente el ítem, cumple parcialmente o no cumple con lo mínimo esperado. Finalmente, todos los descuentos serán asignados manualmente por el cuerpo docente respetando lo expuesto en **el documento de bases generales**.

Para terminar, si durante la realización de tu tarea se te presenta algún problema o situación que pueda afectar tu rendimiento, no dudes en contactar al ayudante de Bienestar de tu sección. El correo está en el **siguiente enlace**.

11. Restricciones y alcances

- Esta tarea es **estrictamente individual**, y está regida por el **Código de honor de Ingeniería**.
- Tu programa debe ser desarrollado en Python 3.11.X con X mayor o igual a 7.
- Tu programa debe estar compuesto por uno o más archivos de extensión `.py` que estén correctamente ordenados por carpeta. **No se revisará archivos en otra extensión como** `.ipynb`.
- Toda el código entregado debe estar contenido en la carpeta y rama (*branch*) indicadas al inicio del enunciado. Ante cualquier problema relacionado a esto, es decir, una carpte distinta a `T2` o una rama distinta a `main`, se recomienda preguntar en las ***issues* del foro**.
- Si no se encuentra especificado en el enunciado, supón que el uso de cualquier librería Python está prohibido. Pregunta en la *issue* especial del **foro** si es que es posible utilizar alguna librería en particular.
- Debes adjuntar un **único archivo markdown**, llamado `README.md`, **conciso y claro**, donde describas los alcances de tu programa, cómo correrlo, las librerías usadas, los supuestos hechos, y las referencias a código externo. El no incluir este archivo, incluir un readme vacío o el subir más de un archivo `.md`, conllevará un **descuento** en tu nota.
- Esta tarea se debe desarrollar **exclusivamente** con los contenidos liberados al momento de publicar el enunciado. No se permitirá utilizar contenidos que se vean posterior a la publicación de esta evaluación.

- Se encuentra estrictamente prohibido citar código que haya sido publicado **después de la liberación del enunciado**. En otras palabras, solo se permite citar contenido que ya exista previo a la publicación del enunciado. Además, se encuentra estrictamente prohibido el uso de herramientas generadoras de código para el apoyo de la evaluación.
- Cualquier aspecto no especificado queda a tu criterio, siempre que no pase por sobre otro que sí sea especificado por enunciado.

Las tareas que no cumplan con las restricciones del enunciado obtendrán la calificación mínima (1,0)**.**